dotBo

BOT Chain Name Service

Whitepaper v1.0 — October 2026

dotBo is a decentralised name service built natively on BOT Chain, enabling human-readable .bot identifiers to resolve to wallet addresses, smart contracts, IPFS content, and arbitrary onchain records. This paper describes the protocol architecture, contract design, economic model, and roadmap.

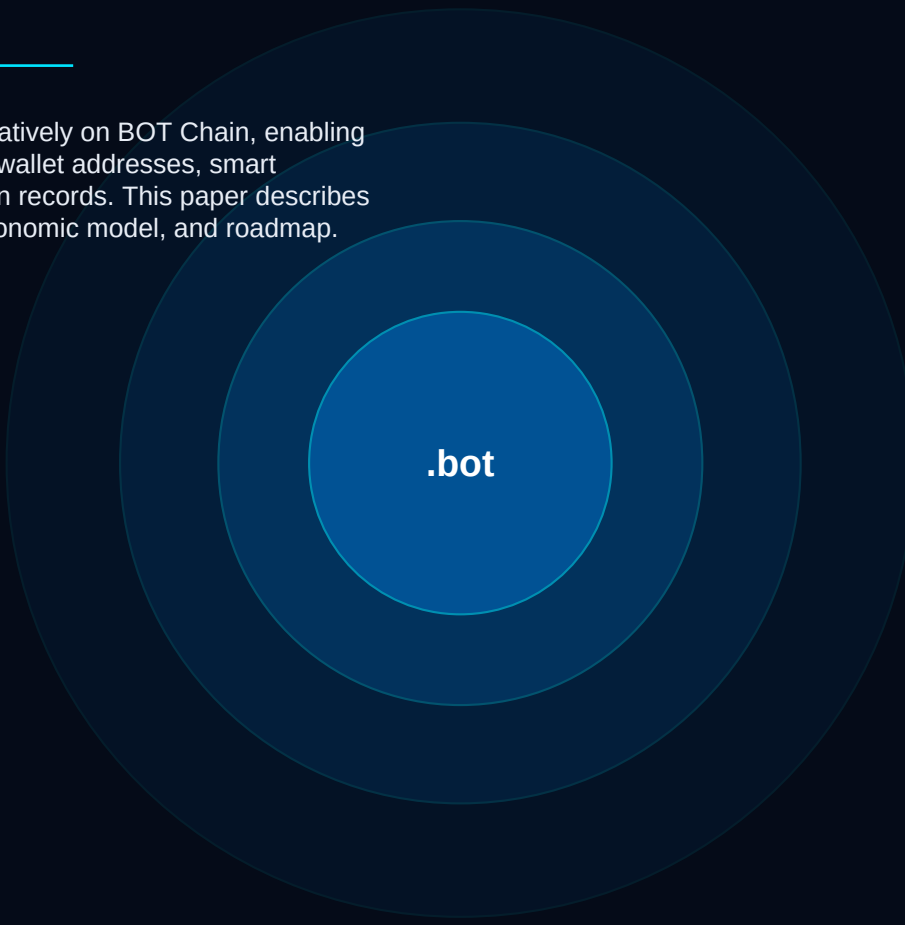

Table of Contents

1. Introduction & Motivation
2. Protocol Overview
3. Technical Architecture
4. Contract Reference
5. Name Lifecycle
6. Onchain Records
7. Subname System
8. Economic Model & Pricing
9. Security Considerations
10. Governance
11. Roadmap
12. Conclusion

1. Introduction & Motivation

The blockchain ecosystem has long grappled with the usability gap between human cognition and machine-readable addresses. A 42-character hexadecimal string is not an identity — it is a technical artefact. dotBo resolves this friction for the BOT Chain ecosystem by providing a fully onchain name service anchored to the .bot top-level domain.

Inspired by the Ethereum Name Service (ENS) and its successors, dotBo is purpose-built for BOT Chain: leveraging its fast finality, low transaction costs, and native developer community. Rather than bridging a foreign name system, dotBo deploys sovereign contracts that treat BOT Chain as the canonical layer of truth.

1.1 The Problem

- Wallet addresses are opaque 42-character strings with no intrinsic meaning.
- Sending funds to the wrong address is irreversible and commonplace.
- Protocol and dApp identity is fragmented across off-chain registries.
- Developers building on BOT Chain lack a native resolution primitive.

1.2 The Solution

dotBo provides a commit-reveal name registry on BOT Chain where any participant can claim a .bot name, associate it with one or more addresses across chains, and publish arbitrary verifiable records — all readable by any contract or frontend without centralised infrastructure.

2. Protocol Overview

dotBo is a three-contract system with clear separation of concerns: ownership, resolution, and registration logic are maintained in distinct contracts that communicate through well-defined interfaces.

Contract	Role	Key Methods
BotRegistry	Canonical ownership & resolver store	setOwner, setResolver, ownerOf
BotResolver	Stores records (addr, text, contenthash)	setAddr, setText, setRecords (batch)
BotRegistrar	Commit-reveal, pricing, renewals, transfers	commit, register, renew, transfer, issueSubname

This separation means the resolver can be upgraded without affecting name ownership, and the registrar logic (pricing, commit windows) can be replaced by governance vote without touching the canonical ownership store.

2.1 Namehash

dotBo follows the EIP-137 namehash algorithm adapted for the .bot TLD. Every name resolves to a deterministic 32-byte node identifier:

```
node("alice.bot") = keccak256(botNode || keccak256("alice")) botNode = keccak256(0x00...00 || keccak256("bot"))
```

This ensures that the full hierarchy is collision-resistant and that subnames (e.g. dev.alice.bot) share a cryptographic parent-child relationship.

3. Technical Architecture

3.1 BotRegistry

The BotRegistry is the immutable root of the system. It maps each namehash node to an owner address, a resolver address, and a TTL value. No name data is stored here — only authorisation. The registry is the single source of truth for: who owns a name, which resolver to consult for its records.

Key design decisions:

- Operators: owners can approve operator addresses (e.g. a dApp frontend) to manage their names.
- Registrar-gated writes: only the approved registrar can create new nodes.
- No upgradeability: the registry is intentionally immutable. Future versions deploy a new registry.

3.2 BotResolver

The BotResolver stores the actual data associated with each name. It supports the following record categories:

`addr(node)`	Primary BOT Chain wallet address (SLIP-44 coin type 60)
`coinAddress(node, coinType)`	Address bytes for any SLIP-44 coin type (ETH, BTC, CELO, etc.)
`text(node, key)`	Arbitrary text record (email, twitter, github, bio, url, avatar, notice…)
`contenthash(node)`	Content hash bytes (IPFS CIDv1, IPNS, Swarm, or custom codec)

The setRecords() batch method allows a name owner to set the wallet address and multiple text records in a single transaction, reducing gas overhead on registration.

3.3 BotRegistrar

The BotRegistrar is the user-facing contract. It enforces the registration lifecycle, commit-reveal scheme, pricing, and subname delegation. It is the only contract authorised to call BotRegistry.setOwner() for new registrations.

4. Contract Reference

4.1 Deployment Sequence

1. Deploy BotRegistry → save registryAddr
2. Deploy BotResolver(registryAddr) → save resolverAddr
3. Deploy BotRegistrar(registryAddr, resolverAddr) → save registrarAddr
4. Call BotRegistry.setRegistrar(registrarAddr) to authorise it

4.2 Label Validation

Labels must satisfy the following rules enforced in BotNamehash.isValidLabel():

```
- Length: 3 to 64 characters - Characters: lowercase a-z, digits 0-9, hyphen (-) - Cannot start or end with a hyphen - No uppercase, no dots, no underscores
```

4.3 Commit-Reveal Scheme

To prevent transaction front-running (where a miner or searcher copies a pending registration and submits it with higher gas), dotBo uses a two-step commit-reveal:

- **Step 1 — Commit:** the user submits keccak256(label || owner || secret), where secret is a random bytes32. The commitment timestamp is recorded.
- **Step 2 — Register:** after at least 60 seconds and within 24 hours of the commit, the user reveals label, owner, and secret. The contract verifies the hash and proceeds with registration.
- If anyone tries to front-run the reveal, they cannot produce a valid commitment hash without the secret.

5. Name Lifecycle

`Available`	No entry in the registry. Open for registration.
`Committed`	Commit hash submitted. Waiting for reveal window (60 s – 24 h).
`Active`	Registered and within the lease period. Owner has full control.
`Grace Period`	Lease has expired but the name is within the 90-day grace window. Only the current owner can renew.
`Released`	Grace period elapsed. Name re-enters the open market.

The grace period is a deliberate protection against accidental expiry. Owners who miss their renewal deadline by less than 90 days retain the right to reclaim their name before it becomes available to others.

5.1 Transfer

A name can be transferred to any address at any time while it is active. Transfer updates both BotRegistry.ownerOf() and the internal BotRegistrar ownership record. Records in BotResolver are not cleared on transfer — the new owner inherits them and can update or clear them independently.

5.2 Renewal

Any address can renew a name (pay on behalf of the owner). Renewal extends from the current expiry date or from block.timestamp if the name is in the grace period. Maximum duration per single renewal is 10 years.

6. Onchain Records

dotBo supports the following standard text record keys, aligned with the ENS text record standard (EIP-634) to maximise compatibility with existing tooling:

`email`	Contact email address
`url`	Primary website URL
`avatar`	Avatar image — IPFS URI, data URI, or HTTPS URL
`description`	Short description or bio (plain text)
`notice`	Important notice displayed to transacting parties
`twitter`	Twitter / X handle
`github`	GitHub username
`telegram`	Telegram username
`discord`	Discord username or server invite
`farcaster`	Farcaster FID or username
`custom.*`	Any key prefixed with "custom." is reserved for user-defined records

Records are gaslessly readable by any contract via resolver.text(node, key). Frontends and dApps are encouraged to implement resolver fallbacks — if a key is not set, the resolver returns an empty string rather than reverting.

7. Subname System

Any active .bot name owner can issue subnames of arbitrary depth. Subnames are full names in their own right — they have an owner, a resolver, and can hold their own records.

Examples:

```
dev.jerydam.bot → developer alias api.myprotocol.bot → smart contract endpoint
v2.myprotocol.bot → versioned deployment
```

Key rules:

- Subnames are issued and revoked by the parent name owner.
- Revocation by the parent clears the registry entry (sets owner to address(0)).
- Subname owners can set their own resolver records independently.
- Subnames do not pay registration fees — they inherit the parent's lease period semantics.
- If a parent name expires and enters the released state, all subnames also lose their authority.

8. Economic Model & Pricing

Registration fees are paid in native BOT and are proportional to name length and lease duration. Shorter names command a premium reflecting their scarcity and desirability.

Label Length	Annual Price	Example	Target
3 chars	100 BOT / yr	dev.bot	Premium short handles
4 chars	20 BOT / yr	defi.bot	Brand names
5+ chars	5 BOT / yr	jerydam.bot	General registrations

Fee calculation:

```
fee = (annualPrice * duration) / 365 days Example: registering jerydam.bot (7 chars) for 2 years: fee = (5 BOT * 730 days) / 365 days = 10 BOT
```

8.1 Fee Destination

All registration and renewal fees accumulate in the BotRegistrar contract. The admin address (initially the deployer, later a governance multisig) can call withdraw() to move funds. Fee allocation is governed by the DAO from Phase 3.

Planned allocation (post-DAO):

50%	Protocol treasury — funds ongoing development and grants
30%	Liquidity and ecosystem incentives
20%	Burned or redistributed to .bot name holders as governance weight

9. Security Considerations

9.1 Front-running Protection

The commit-reveal scheme with a mandatory 60-second window eliminates profitable front-running. An attacker who observes the commit transaction cannot register the name without the secret. An attacker who observes the reveal has already missed the 60-second window by definition in a single block.

9.2 Resolver Trust

The BotResolver is trusted by downstream consumers only to the extent that the registry owner has set it. Users should verify resolver integrity before consuming records in high-value contexts. A future on-chain resolver whitelist may be proposed through governance.

9.3 Registry Immutability

BotRegistry is not upgradeable. This is intentional: the ownership record is the root of trust, and upgradeability would introduce an attack vector. Protocol improvements are made via new registrar and resolver deployments that the registry delegates to.

9.4 Admin Key

The deployer admin key can pause the registrar (halt new registrations) and withdraw fees. It cannot rewrite ownership records. The admin key will be transferred to a multisig wallet and eventually a DAO timelock as the protocol matures.

9.5 Grace Period Semantics

During the 90-day grace period, only the existing owner can renew. This prevents squatting on recently expired names while giving owners sufficient time to act.

10. Governance

dotBo governance evolves in phases alongside the protocol. The initial admin key is necessary for rapid iteration; decentralisation is a first-class goal.

Phase 1–2	Admin multisig (3-of-5). Signers are founding team members.
Phase 3	DAO token launch. Governance token weighted by name ownership and tenure.
Phase 4	On-chain proposals and timelock. Minimum 48h execution delay for parameter changes.
Long-term	Immutable core (registry), governance-controlled periphery (registrar, pricing).

Governable parameters include registration fees, grace period duration, the approved registrar address, treasury allocation ratios, and the fee recipient. The registry itself is permanently immutable.

11. Roadmap

Phase	Timeline	Deliverables
Phase 1 — Foundation	Q4 2026	Registry, Resolver, Registrar contracts on BOT Chain testnet; commit-reveal launch; basic frontend
Phase 2 — Growth	Q1 2027	Mainnet launch; subname issuance; batch record-setting; SDK release; resolver integrations
Phase 3 — Ecosystem	Q2 2027	dApp resolver SDK; wallet integrations; BOT Chain explorer .bot resolution; DAO governance token
Phase 4 — Interop	Q3 2027	Cross-chain resolution; EIP-137 compatibility layer; mobile app; community grants programme

12. Conclusion

dotBo fills a foundational gap in the BOT Chain ecosystem. Human-readable identity is not a luxury feature — it is a prerequisite for mass adoption. By building natively on BOT Chain with a clean three-contract architecture, commit-reveal anti-frontrunning, rich onchain records, and a clear path to community governance, dotBo provides the identity layer that BOT Chain developers and users need today.

The protocol is deliberately minimal in its first version. The registry is immutable. The resolver is simple and extensible. The registrar encodes sensible defaults while remaining replaceable. This conservatism is a feature: the identity layer is load-bearing infrastructure, and stability matters more than novelty.

We invite the BOT Chain community to review the contracts, propose improvements, and join the effort to build the onchain identity primitive that the ecosystem deserves.

Website	https://dotbo.bot
Contracts	github.com/dotbo/contracts
Docs	docs.dotbo.bot
Discord	discord.gg/dotbo
Version	Whitepaper v1.0 — October 2026